

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 11

Persistencia local con SQLite (sqflite): crear la base y guardar datos

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles	SEMESTRE 6.º · Mención Ing. de Software
TEMA Base de datos local con sqflite	DURACIÓN 5 horas (1 sesión de laboratorio)
CAPÍTULO Cap. 5 — Programación asíncrona y SQLite	MODALIDAD Laboratorio · individual
DOCENTE M. Sc. Lic. Huáscar Fedor Gonzales Guzmán	REQUISITO Guía de Laboratorio N.º 10

NOTA · Del estado en memoria a los datos persistentes

Todas las apps que construiste hasta ahora **olvidan todo** al cerrarse: las tareas, el carrito, los estudiantes registrados vivían en una [List](#) en memoria. En esta guía guardarás los datos en una base de datos **SQLite** dentro del propio dispositivo, de modo que sobrevivan a cierres y reinicios.

01 Objetivos

Objetivo general. Incorporar una base de datos SQLite local en una app Flutter mediante el paquete sqflite, creando su esquema e implementando las operaciones de inserción y consulta.

Objetivos específicos:

- Agregar las dependencias [sqflite](#) y [path](#) al proyecto.
- Abrir la base con [openDatabase](#) y crear la tabla en [onCreate](#).
- Convertir objetos a filas y viceversa con [toMap\(\)](#) y [fromMap\(\)](#).
- Centralizar el acceso a datos en una clase [DbHelper](#) con una única conexión.
- Insertar y listar registros, y comprobar que los datos persisten al reiniciar la app.

02 Marco teórico

DEFINICIÓN · SQLite

Motor de base de datos relacional **embebido**: no hay servidor, toda la base es un único archivo dentro del almacenamiento de la app. Usa SQL estándar ([CREATE](#), [INSERT](#), [SELECT](#)...) y viene incluido en Android e iOS.

DEFINICIÓN · sqflite y path

`sqflite` es el plugin que conecta Flutter con SQLite; todas sus operaciones son **asíncronas** (devuelven `Future`). `path` ayuda a construir la ruta del archivo de forma correcta en cada sistema operativo con `join()`.

DEFINICIÓN · Filas como Map

`sqflite` no conoce tus clases: inserta y devuelve filas como `Map<String, Object?>`, donde cada clave es el nombre de una columna. Por eso cada modelo necesita `toMap()` (objeto → fila) y `fromMap()` (fila → objeto).

Tipos de SQLite y su equivalente en Dart:

SQLite	Dart	Observación
INTEGER	int	También se usa para bool (0 = false, 1 = true)
REAL	double	Montos, precios, notas con decimales
TEXT	String	Fechas en formato ISO: 2026-09-27
NULL	null	Ausencia de valor (id antes de insertar)

03**Requisitos previos**

Requisito indispensable. Guía 11 completada (`Future`, `async/await`, `FutureBuilder`) y conocimientos básicos de SQL.

Conceptos que se reutilizan:`async / await``FutureBuilder``TextEditingController``mounted``SnackBar`**ATENCIÓN · Plataforma de ejecución: usa el emulador Android**

`sqflite` funciona en **Android**, **iOS** y **macOS**. No funciona en Flutter web, y en Windows de escritorio requiere el paquete adicional `sqflite_common_ffi`. Para estas prácticas ejecuta siempre en el **emulador Android** configurado en la Guía 2 (o en un teléfono físico).

04**Desarrollo · Parte A — Dependencias, modelo y DbHelper****PASO 1 Crear el proyecto y agregar dependencias**

1. Crea el proyecto `notas_app` y, desde su carpeta raíz, agrega los dos paquetes:

```

• • • PowerShell

flutter create notas_app
cd notas_app
flutter pub add sqflite path

```

2. Verifica que `pubspec.yaml` contenga algo similar (las versiones pueden variar):

• • • `pubspec.yaml` · YAML

```
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.4.2
  path: ^1.9.1
```

3. Organiza el código en carpetas; a partir de esta guía cada responsabilidad va en su propio archivo:

• • • Estructura del proyecto

```
lib/
├── main.dart           // interfaz
├── models/
│   └── nota.dart       // modelo de datos
├── db/
│   └── db_helper.dart  // acceso a SQLite
```

PASO 2 El modelo Nota con `toMap` y `fromMap`

1. Crea `lib/models/nota.dart`. El `id` es opcional (`int?`) porque lo asigna SQLite al insertar:

• • • `lib/models/nota.dart` · Dart

```
class Nota {
  final int? id;
  final String titulo;
  final String contenido;
  final String fecha;

  const Nota({
    this.id,
    required this.titulo,
    required this.contenido,
    required this.fecha
  });

  Map<String, Object?> toMap() {
    return {
      if (id != null) 'id': id,
      'titulo': titulo,
      'contenido': contenido,
      'fecha': fecha
    };
  }

  factory Nota.fromMap(Map<String, Object?> map) {
    return Nota(
      id: map['id'] as int?,
      titulo: map['titulo'] as String,
      contenido: map['contenido'] as String,
      fecha: map['fecha'] as String,
    );
  }
}
```

NOTA · Las claves del Map son los nombres de las columnas

Si en `toMap()` escribes `'titulo'` pero la columna se llama `title`, la inserción fallará. Mantén los nombres idénticos en el modelo y en el `CREATE TABLE`.

PASO 3 La clase DbHelper

1. Crea `lib/db/db_helper.dart`. Es un **singleton**: existe una sola instancia y una sola conexión abierta para toda la app.

• • • lib/db/db_helper.dart · Dart

```
import 'package:flutter/foundation.dart';
import 'package:notas_app/models/nota.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class DbHelper {
  DbHelper._();
  static final DbHelper instancia = DbHelper._();

  static Database? _db;

  Future<Database> get database async {
    _db ??= await _abrir();
    return _db!;
  }

  Future<Database> _abrir() async {
    final ruta = join(await getDatabasesPath(), 'notas.db');
    debugPrint(ruta);
    return openDatabase(
      ruta,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE notas(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            titulo TEXT NOT NULL,
            contenido TEXT NOT NULL,
            fecha TEXT NOT NULL
          )
        ''');
      },
    );
  }

  Future<int> insertarNota(Nota nota) async {
    final db = await database;
    return db.insert('notas', nota.toMap());
  }

  Future<List<Nota>> obtenerNotas() async {
    final db = await database;
    final filas = await db.query('notas', orderBy: 'id DESC');
    return filas.map((f) => Nota.fromMap(f)).toList();
  }

  Future<int> contarNotas() async {
    final db = await database;
    final res = await db.rawQuery('SELECT COUNT(*) FROM notas');
    return Sqflite.firstIntValue(res) ?? 0;
  }
}
```

}

DEFINICIÓN · Cómo funciona openDatabase

La primera vez que se abre la base, el archivo `notas.db` no existe: SQLite lo crea y ejecuta `onCreate` con la `version` indicada. En las siguientes aperturas el archivo ya existe y `onCreate` **no vuelve a ejecutarse**. El operador `??=` garantiza que la base se abra una sola vez.

ATENCIÓN · Cambiaste el CREATE TABLE y no pasa nada

Como `onCreate` solo corre al crear el archivo, modificar la tabla después no tiene efecto. Durante el desarrollo **desinstala la app** del emulador para empezar de cero (en la Guía 15 aprenderás a migrar con `onUpgrade`).

05**Desarrollo · Parte B — Guardar y listar desde la interfaz****PASO 4 La pantalla de notas**

1. Reemplaza `lib/main.dart`. Arriba, un formulario sencillo; abajo, la lista leída de la base con el patrón `FutureBuilder` de la Guía 11:

```
• • • lib/main.dart · Dart (1 de 2)

import 'package:flutter/material.dart';
import 'db/db_helper.dart';
import 'models/nota.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Notas',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.cyan),
      home: const PantallaNotas(),
    );
  }
}

class PantallaNotas extends StatefulWidget {
  const PantallaNotas({super.key});

  @override
  State<PantallaNotas> createState() => _PantallaNotasState();
}

class _PantallaNotasState extends State<PantallaNotas> {
  final _db = DbHelper.instancia;
  final _tituloCtrl = TextEditingController();
  final _contenidoCtrl = TextEditingController();
  late Future<List<Nota>> _futuroNotas;
```

```

@override
void initState() {
  super.initState();
  _futuroNotas = _db.obtenerNotas();
}

@override
void dispose() {
  _tituloCtrl.dispose();
  _contenidoCtrl.dispose();
  super.dispose();
}

Future<void> _guardar() async {
  final titulo = _tituloCtrl.text.trim();
  if (titulo.isEmpty) return;
  final nota = Nota(
    titulo: titulo,
    contenido: _contenidoCtrl.text.trim(),
    fecha: DateTime.now().toIso8601String().substring(0, 10),
  );
  final id = await _db.insertarNota(nota);
  _tituloCtrl.clear();
  _contenidoCtrl.clear();
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Nota guardada con id $id')),
  );
  setState(() {
    _futuroNotas = _db.obtenerNotas();
  });
}

```

2. Completa la clase con el método build:

• • • lib/main.dart · Dart (2 de 2)

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Mis notas')),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.all(12),
          child: Column(
            children: [
              TextField(
                controller: _tituloCtrl,
                decoration: const InputDecoration(labelText: 'Titulo'),
              ),
              const SizedBox(height: 8),
              TextField(
                controller: _contenidoCtrl,
                decoration: const InputDecoration(labelText: 'Contenido'),
                maxLines: 2,
              ),
              const SizedBox(height: 12),
              SizedBox(
                width: double.infinity,
                child: FilledButton.icon(
                  onPressed: _guardar,
                  icon: const Icon(Icons.save),
                ),
              ),
            ],
          ),
        ),
      ],
    ),
  );
}

```


06

Métodos de sqflite de uso frecuente

Método	SQL equivalente	Devuelve
db.insert(tabla, map)	INSERT INTO ...	int — id de la fila nueva
db.query(tabla, ...)	SELECT ... FROM ...	List<Map<String, Object?>>
db.update(tabla, map, where:)	UPDATE ... SET ... WHERE	int — filas modificadas
db.delete(tabla, where:)	DELETE FROM ... WHERE	int — filas eliminadas
db.rawQuery(sql, args)	Cualquier SELECT	List<Map<String, Object?>>
db.execute(sql)	CREATE, ALTER, PRAGMA	void

En esta guía usaste `insert`, `query`, `rawQuery` y `execute`. En la Guía 13 completarás el CRUD con `update` y `delete`.

07

Verificación de resultados y entregable

Marca cada punto como verificación de que completaste la práctica:

- ☐ sqflite y path están en pubspec.yaml y el proyecto compila.
- ☐ El código está separado en models/, db/ y main.dart.
- ☐ Nota implementa toMap() y fromMap() con los nombres de columna correctos.
- ☐ DbHelper abre una sola conexión y crea la tabla en onCreate.
- ☐ Las notas se guardan y aparecen en la lista con su id.
- ☐ Las notas siguen visibles tras detener y volver a ejecutar la app.

ENTREGA · Qué entregar

La carpeta `lib/` completa y un documento (PDF o Word) con: (1) captura con al menos tres notas guardadas; (2) captura tras reiniciar la app mostrando que persisten; y (3) **reto personal**: muestra el total de notas en el `AppBar` usando `contarNotas()` (pista: otro `FutureBuilder`, o guarda el conteo en el estado). Explica tu solución en 3–5 líneas.

08

Cuestionario

- ¿Qué ventajas tiene SQLite frente a guardar los datos en una lista en memoria?
- ¿Por qué el id del modelo se declara como int? (anulable)?
- ¿Cuándo se ejecuta onCreate y qué pasa si luego modificas el CREATE TABLE?
- ¿Qué problema evita que DbHelper sea un singleton con una sola conexión?
- ¿Por qué todas las operaciones de sqflite devuelven un Future?

09

Rúbrica de evaluación

Criterio	Puntaje
Dependencias y estructura del proyecto en capas	15 %
Modelo con toMap() / fromMap() correctos	20 %
DbHelper: apertura única, onCreate e inserción/consulta	30 %
Interfaz que guarda y lista; persistencia comprobada	20 %
Cuestionario resuelto	15 %

10

Referencias

- Flutter Cookbook — Persist data with SQLite. docs.flutter.dev/cookbook/persistence/sqlite
- Paquete sqflite en pub.dev. pub.dev/packages/sqflite
- Paquete path en pub.dev. pub.dev/packages/path
- Tipos de datos en SQLite. sqlite.org/datatype3.html

Contenido basado en la documentación oficial de Flutter (v3.44) y el paquete sqflite (^2.4.x). Elaborado para INF 662 — UATF.